```
# Install required libraries
!pip install nltk rouge-score datasets
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-
Requirement already satisfied: rouge-score in /usr/local/lib/python3.1
Requirement already satisfied: datasets in /usr/local/lib/python3.12/d
Requirement already satisfied: click in /usr/local/lib/python3.12/dist
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dis
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/pytho
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/di
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist
Requirement already satisfied: six>=1.14.0 in /usr/local/lib/python3.1
Requirement already satisfied: filelock in /usr/local/lib/python3.12/d
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/pytho
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/py
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dis
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/pyth
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dis
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/loc
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/l
Requirement already satisfied: packaging in /usr/local/lib/python3.12/
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.1
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/loca
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/pyth
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/py
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/py
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/li
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/l
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/pyth
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/pyt
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/p
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/pyth
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/pyt
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/py
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/l
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12
```

```
# Core libraries
import numpy as np
import tensorflow as tf
```

```
import random
import time
import os

# NLP libraries
import nltk
from nltk.translate.bleu_score import sentence_bleu, corpus_bleu, Smo

# Visualization
import matplotlib.pyplot as plt

# Dataset (for Part 3)
from datasets import load_dataset
```

```
# Download tokenizer
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
True
```

```
# Fix randomness for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)
```

```
print("TensorFlow version:", tf.__version__)

gpus = tf.config.list_physical_devices('GPU')
print("GPU available:", gpus)

if gpus:
    print("GPU is enabled 🚀")
else:
    print("Running on CPU ⚠️")
```

```
TensorFlow version: 2.19.0
GPU available: [PhysicalDevice(name='/physical_device:GPU:0', device_t
GPU is enabled 🚀
```

```
print("All libraries installed and imported successfully ✅")
```

```
All libraries installed and imported successfully ✅
```

```
# Small English → German dataset (toy dataset)

english_sentences = [
    "i love you",
    "hello world",
    "how are you",
    "good morning",
    "thank you",
```

```
    "i am happy",
    "he is good",
    "she is sad",
    "they are here",
    "i am learning"
]

german_sentences = [
    "ich liebe dich",
    "hallo welt",
    "wie geht es dir",
    "guten morgen",
    "danke schoen",
    "ich bin gluecklich",
    "er ist gut",
    "sie ist traurig",
    "sie sind hier",
    "ich lerne"
]

print("Dataset loaded successfully ✅")
print("Total sentence pairs:", len(english_sentences))
```

```
Dataset loaded successfully ✅
Total sentence pairs: 10
```

```
# Display sample pairs

for i in range(5):
    print(f"{i+1}. EN: {english_sentences[i]}  →  DE: {german_sentenc
```

```
1. EN: i love you  →  DE: ich liebe dich
2. EN: hello world  →  DE: hallo welt
3. EN: how are you  →  DE: wie geht es dir
4. EN: good morning  →  DE: guten morgen
5. EN: thank you  →  DE: danke schoen
```

```
# Simple lowercase cleaning

def clean_text(sentence):
    return sentence.lower().strip()

english_sentences = [clean_text(s) for s in english_sentences]
german_sentences = [clean_text(s) for s in german_sentences]

print("Text cleaned successfully ✅")
```

```
Text cleaned successfully ✅
```

```
# Analyze sentence lengths

max_len_en = max(len(s.split()) for s in english_sentences)
max_len_de = max(len(s.split()) for s in german_sentences)
```

```
print("Max English length:", max_len_en)
print("Max German length:", max_len_de)
```

```
Max English length: 3
Max German length: 4
```

```
# Special tokens
PAD = 0
SOS = 1
EOS = 2
UNK = 3

class Vocabulary:
    def __init__(self):
        self.word2idx = {
            "<pad>": PAD,
            "<sos>": SOS,
            "<eos>": EOS,
            "<unk>": UNK
        }
        self.idx2word = {
            PAD: "<pad>",
            SOS: "<sos>",
            EOS: "<eos>",
            UNK: "<unk>"
        }
        self.count = 4  # starting index

    def add_sentence(self, sentence):
        for word in sentence.split():
            self.add_word(word)

    def add_word(self, word):
        if word not in self.word2idx:
            self.word2idx[word] = self.count
            self.idx2word[self.count] = word
            self.count += 1

    def __len__(self):
        return self.count
```

```
# Create vocab objects
src_vocab = Vocabulary()
tgt_vocab = Vocabulary()

# Add sentences to vocab
for sentence in english_sentences:
    src_vocab.add_sentence(sentence)

for sentence in german_sentences:
    tgt_vocab.add_sentence(sentence)

print("Source Vocabulary Size:", len(src_vocab))
print("Target Vocabulary Size:", len(tgt_vocab))
```

```
Source Vocabulary Size: 23
Target Vocabulary Size: 27
```

```
# Show some word mappings

print("\nSample word mappings (English):")
for word, idx in list(src_vocab.word2idx.items())[:10]:
    print(f"{word} → {idx}")

print("\nSample word mappings (German):")
for word, idx in list(tgt_vocab.word2idx.items())[:10]:
    print(f"{word} → {idx}")
```

```

Sample word mappings (English):
<pad> → 0
<sos> → 1
<eos> → 2
<unk> → 3
i → 4
love → 5
you → 6
hello → 7
world → 8
how → 9

Sample word mappings (German):
<pad> → 0
<sos> → 1
<eos> → 2
<unk> → 3
ich → 4
liebe → 5
dich → 6
hallo → 7
welt → 8
wie → 9
```

```
# Convert sentence to indices

def encode_sentence(sentence, vocab):
    return [vocab.word2idx.get(word, UNK) for word in sentence.split(
```

```
# Example
print("Sample Encoding:")
print("Sentence:", english_sentences[0])
print("Encoded:", encode_sentence(english_sentences[0], src_vocab))
```

```
Sample Encoding:
Sentence: i love you
Encoded: [4, 5, 6]
```

```
# Add <sos> and <eos>

def add_tokens(sequence):
    return [SOS] + sequence + [EOS]

# Example
sample = encode_sentence(english_sentences[0], src_vocab)
print("With tokens:", add_tokens(sample))
```

```
With tokens: [1, 4, 5, 6, 2]
```

```
# Pad sequences to fixed length

def pad_sequence(seq, max_len):
    return seq + [PAD] * (max_len - len(seq))
```

```
MAX_LEN = 10

encoder_inputs = []
decoder_inputs = []
decoder_targets = []

for src, tgt in zip(english_sentences, german_sentences):

    # Encode
    src_seq = encode_sentence(src, src_vocab)
    tgt_seq = encode_sentence(tgt, tgt_vocab)

    # Add tokens
    src_seq = add_tokens(src_seq)
    tgt_seq = add_tokens(tgt_seq)

    # Prepare decoder input and target
    dec_input = tgt_seq[:-1]   # remove last
    dec_target = tgt_seq[1:]   # remove first

    # Pad
    encoder_inputs.append(pad_sequence(src_seq, MAX_LEN))
    decoder_inputs.append(pad_sequence(dec_input, MAX_LEN))
    decoder_targets.append(pad_sequence(dec_target, MAX_LEN))

# Convert to numpy
import numpy as np
```

```
encoder_inputs = np.array(encoder_inputs)
decoder_inputs = np.array(decoder_inputs)
decoder_targets = np.array(decoder_targets)

print("Shapes:")
print("Encoder:", encoder_inputs.shape)
print("Decoder Input:", decoder_inputs.shape)
print("Decoder Target:", decoder_targets.shape)
```

```
Shapes:
Encoder: (10, 10)
Decoder Input: (10, 10)
Decoder Target: (10, 10)
```

```
# Show one sample

print("Original:", english_sentences[0])
print("Encoded:", encoder_inputs[0])

print("\nDecoder Input:", decoder_inputs[0])
print("Decoder Target:", decoder_targets[0])
```

```
Original: i love you
Encoded: [1 4 5 6 2 0 0 0 0 0]

Decoder Input: [1 4 5 6 0 0 0 0 0 0]
Decoder Target: [4 5 6 2 0 0 0 0 0 0]
```

```
class Encoder(tf.keras.layers.Layer):
    def __init__(self, vocab_size, embedding_dim=64, hidden_dim=128):
        super().__init__()

        # Embedding layer: converts word indices → dense vectors
        self.embedding = tf.keras.layers.Embedding(vocab_size, embedd

        # Bidirectional LSTM
        self.bilstm = tf.keras.layers.Bidirectional(
            tf.keras.layers.LSTM(hidden_dim,
                                 return_sequences=True,
                                 return_state=True)
        )

        # Dense layer to reduce dimension (important!)
        self.fc = tf.keras.layers.Dense(hidden_dim)

    def call(self, x):
        # x shape: (batch_size, seq_len)

        x = self.embedding(x)
        # shape: (batch, seq_len, embedding_dim)

        outputs, forward_h, forward_c, backward_h, backward_c = self.

        # Concatenate forward & backward states
```

```
        hidden = tf.concat([forward_h, backward_h], axis=-1)
        cell = tf.concat([forward_c, backward_c], axis=-1)

        # Reduce dimension to match decoder
        hidden = self.fc(hidden)
        cell = self.fc(cell)

        return outputs, hidden, cell
```

```
# Create encoder instance
encoder = Encoder(vocab_size=len(src_vocab.word2idx))

print("Encoder initialized successfully ✅")
```

```
Encoder initialized successfully ✅
```

```
# Test with sample input

sample_input = encoder_inputs[:2]  # batch of 2

outputs, hidden, cell = encoder(sample_input)

print("Encoder Output Shape:", outputs.shape)
print("Hidden Shape:", hidden.shape)
print("Cell Shape:", cell.shape)
```

```
Encoder Output Shape: (2, 10, 256)
Hidden Shape: (2, 128)
Cell Shape: (2, 128)
```

```
class LuongAttention(tf.keras.layers.Layer):
    def __init__(self, hidden_dim):
        super().__init__()

        # Linear layer to transform encoder outputs
        self.W = tf.keras.layers.Dense(hidden_dim)

    def call(self, decoder_hidden, encoder_outputs):
        """
        decoder_hidden: (batch, hidden_dim)
        encoder_outputs: (batch, seq_len, hidden_dim)
        """

        # Expand decoder hidden → (batch, 1, hidden)
        decoder_hidden = tf.expand_dims(decoder_hidden, 1)

        # Compute score
        score = tf.reduce_sum(
            self.W(encoder_outputs) * decoder_hidden,
            axis=2
        )  # (batch, seq_len)

        # Convert to probabilities
```

```
        attention_weights = tf.nn.softmax(score, axis=1)

        # Compute context vector
        context_vector = tf.reduce_sum(
            encoder_outputs * tf.expand_dims(attention_weights, 2),
            axis=1
        )  # (batch, hidden_dim)

        return context_vector, attention_weights
```

```
attention = LuongAttention(hidden_dim=128)

print("Attention initialized ✅")
```

```
Attention initialized ✅
```

```
# Use encoder output from previous section

context, weights = attention(hidden, outputs)

print("Context Shape:", context.shape)
print("Attention Weights Shape:", weights.shape)
```

```
Context Shape: (2, 256)
Attention Weights Shape: (2, 10)
```

```
print("Sum of weights (should be ~1):")
print(tf.reduce_sum(weights[0]))
```

```
Sum of weights (should be ~1):
tf.Tensor(1.0, shape=(), dtype=float32)
```

```
class Decoder(tf.keras.layers.Layer):
    def __init__(self, vocab_size, embedding_dim=64, hidden_dim=128):
        super().__init__()

        # Embedding layer
        self.embedding = tf.keras.layers.Embedding(vocab_size, embedd

        # LSTM Cell (step-by-step decoding)
        self.lstm = tf.keras.layers.LSTMCell(hidden_dim)

        # Attention layer
        self.attention = LuongAttention(hidden_dim)

        # Final output layer
        self.fc = tf.keras.layers.Dense(vocab_size)

    def call(self, decoder_inputs, hidden, cell, encoder_outputs):
        """
        decoder_inputs: (batch, seq_len)
        hidden, cell: (batch, hidden)
        encoder_outputs: (batch, seq_len, hidden)
```

```
        """

        outputs = []

        # Loop over each time step
        for t in range(decoder_inputs.shape[1]):

            # Get word at time step t
            x = self.embedding(decoder_inputs[:, t])   # (batch, embe

            # LSTM step
            hidden, [hidden, cell] = self.lstm(x, [hidden, cell])

            # Apply attention
            context_vector, attention_weights = self.attention(hidden

            # Combine hidden + context
            combined = tf.concat([hidden, context_vector], axis=-1)

            # Predict next word
            output = self.fc(combined)

            outputs.append(output)

        # Stack outputs → (batch, seq_len, vocab_size)
        outputs = tf.stack(outputs, axis=1)

        return outputs
```

```
decoder = Decoder(vocab_size=len(tgt_vocab.word2idx))

print("Decoder initialized successfully ✅")
```

```
Decoder initialized successfully ✅
```

```
# Test with encoder output

sample_input = encoder_inputs[:2]

enc_outputs, hidden, cell = encoder(sample_input)

dec_out = decoder(decoder_inputs[:2], hidden, cell, enc_outputs)

print("Decoder Output Shape:", dec_out.shape)
```

```
Decoder Output Shape: (2, 10, 27)
```

```
class Seq2Seq(tf.keras.Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder
```

```
    def call(self, encoder_inputs, decoder_inputs):
        """
        encoder_inputs: (batch, seq_len)
        decoder_inputs: (batch, seq_len)
        """

        # Step 1: Pass through encoder
        encoder_outputs, hidden, cell = self.encoder(encoder_inputs)

        # Step 2: Pass through decoder (with attention)
        decoder_outputs = self.decoder(
            decoder_inputs,
            hidden,
            cell,
            encoder_outputs
        )

        return decoder_outputs
```

```
model = Seq2Seq(encoder, decoder)

print("Full Model Created Successfully ✅")
```

```
Full Model Created Successfully ✅
```

```
# Test forward pass

sample_enc = encoder_inputs[:2]
sample_dec = decoder_inputs[:2]

outputs = model(sample_enc, sample_dec)

print("Model Output Shape:", outputs.shape)
```

```
Model Output Shape: (2, 10, 27)
```

```
class Seq2Seq_NoAttention(tf.keras.Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def call(self, encoder_inputs, decoder_inputs):
        encoder_outputs, hidden, cell = self.encoder(encoder_inputs)

        # ❌ No attention used
        outputs = []

        for t in range(decoder_inputs.shape[1]):
            x = decoder.embedding(decoder_inputs[:, t])
            hidden, [hidden, cell] = decoder.lstm(x, [hidden, cell])
            out = decoder.fc(hidden)
            outputs.append(out)
```

```
        return tf.stack(outputs, axis=1)
```

```
# Loss function
loss_function = tf.keras.losses.SparseCategoricalCrossentropy(from_lo
```

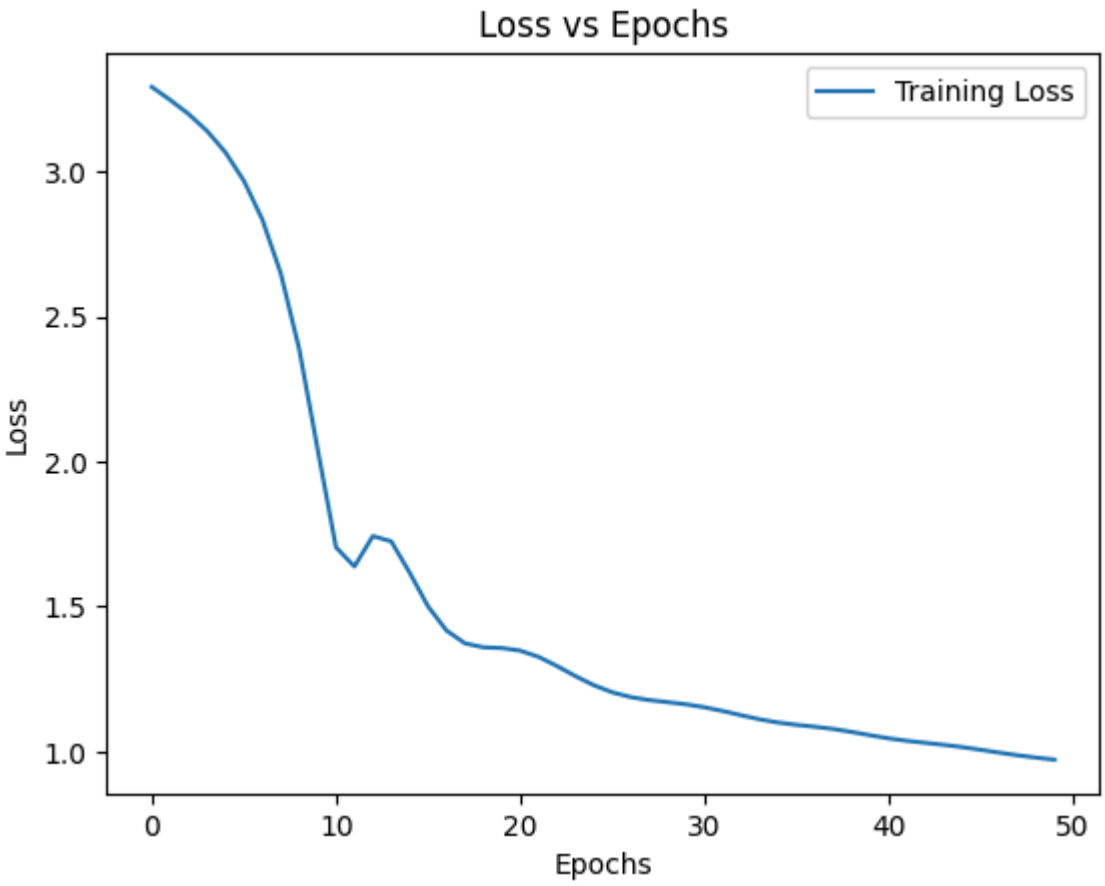
```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

```
@tf.function
def train_step(enc_inp, dec_inp, dec_tar):

    with tf.GradientTape() as tape:
        predictions = model(enc_inp, dec_inp)

        loss = loss_function(dec_tar, predictions)

    gradients = tape.gradient(loss, model.trainable_variables)

    optimizer.apply_gradients(zip(gradients, model.trainable_variable

    return loss
```

```
EPOCHS = 50

loss_history = []

start_time = time.time()

for epoch in range(EPOCHS):

    loss = train_step(encoder_inputs, decoder_inputs, decoder_targets

    loss_history.append(loss.numpy())

    if epoch % 10 == 0:
        print(f"Epoch {epoch} | Loss: {loss.numpy():.4f}")

end_time = time.time()

training_time = end_time - start_time

print("\nTraining Completed ✅")
print("Total Training Time:", training_time, "seconds")
```

```
Epoch 0 | Loss: 3.2909
Epoch 10 | Loss: 1.7047
Epoch 20 | Loss: 1.3485
Epoch 30 | Loss: 1.1533
Epoch 40 | Loss: 1.0461

Training Completed ✅
Total Training Time: 8.017969369888306 seconds
```

```
import matplotlib.pyplot as plt

plt.plot(loss_history, label="Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Loss vs Epochs")
plt.legend()
plt.show()
```




```
class Decoder_NoAttention(tf.keras.layers.Layer):
    def __init__(self, vocab_size, embedding_dim=64, hidden_dim=128):
        super().__init__()

        self.embedding = tf.keras.layers.Embedding(vocab_size, embedd
        self.lstm = tf.keras.layers.LSTMCell(hidden_dim)

        # ⚠️ IMPORTANT: input = hidden only (NOT hidden+context)
        self.fc = tf.keras.layers.Dense(vocab_size)

    def call(self, decoder_inputs, hidden, cell):
        outputs = []

        for t in range(decoder_inputs.shape[1]):
            x = self.embedding(decoder_inputs[:, t])

            hidden, [hidden, cell] = self.lstm(x, [hidden, cell])

            # ONLY hidden (correct)
```

```
            out = self.fc(hidden)

            outputs.append(out)

        return tf.stack(outputs, axis=1)
```

```
class Seq2Seq_NoAttention(tf.keras.Model):
    def __init__(self, encoder, decoder):
        super().__init__()
        self.encoder = encoder
        self.decoder = decoder

    def call(self, encoder_inputs, decoder_inputs):
        encoder_outputs, hidden, cell = self.encoder(encoder_inputs)

        # ⚠️ No attention → ignore encoder_outputs
        outputs = self.decoder(decoder_inputs, hidden, cell)

        return outputs
```

```
decoder_no_attn = Decoder_NoAttention(vocab_size=len(tgt_vocab.word2i

model_no_attn = Seq2Seq_NoAttention(encoder, decoder_no_attn)
```

```
optimizer2 = tf.keras.optimizers.Adam()

loss_no_attn = []

for epoch in range(EPOCHS):

    with tf.GradientTape() as tape:
        preds = model_no_attn(encoder_inputs, decoder_inputs)
        loss = loss_function(decoder_targets, preds)

    grads = tape.gradient(loss, model_no_attn.trainable_variables)
    optimizer2.apply_gradients(zip(grads, model_no_attn.trainable_var

    loss_no_attn.append(loss.numpy())

    if epoch % 10 == 0:
        print(f"[No Attn] Epoch {epoch} | Loss: {loss.numpy():.4f}")
```

```
[No Attn] Epoch 0 | Loss: 3.2585
[No Attn] Epoch 10 | Loss: 1.2695
[No Attn] Epoch 20 | Loss: 1.0323
[No Attn] Epoch 30 | Loss: 0.9180
[No Attn] Epoch 40 | Loss: 0.8302
```

```
def predict_sentence(model, sentence):

    # Encode input sentence
    seq = encode_sentence(sentence, src_vocab)
    seq = [SOS] + seq + [EOS]
```

```
    # Pad input
    seq = seq + [PAD] * (10 - len(seq))
    seq = np.array([seq])

    # Encoder forward pass
    encoder_outputs, hidden, cell = model.encoder(seq)

    # Start with <sos>
    dec_input = np.array([[SOS]])

    result = []

    for _ in range(10):

        preds = model.decoder(dec_input, hidden, cell, encoder_output

        # Get predicted word
        pred_id = tf.argmax(preds[0, 0]).numpy()

        if pred_id == EOS:
            break

        result.append(tgt_vocab.idx2word.get(pred_id, "<unk>"))

        # Next input
        dec_input = np.array([[pred_id]])

    return " ".join(result)
```

```
print("Input:", "i love you")
print("Prediction:", predict_sentence(model, "i love you"))
```

```
Input: i love you
Prediction: ich ich ich ich ich ich ich ich ich ich
```

```
test_sentences = [
    "i love you",
    "good morning",
    "thank you"
]

for sent in test_sentences:
    print("\nInput:", sent)
    print("Prediction:", predict_sentence(model, sent))
```

```
Input: i love you
Prediction: ich ich ich ich ich ich ich ich ich ich

Input: good morning
Prediction: ich ich ich ich ich ich ich ich ich ich

Input: thank you
Prediction: ich ich ich ich ich ich ich ich ich ich
```

```
def calculate_bleu(reference, prediction):
    smoothie = SmoothingFunction().method4
    return sentence_bleu([reference.split()], prediction.split(), smo
```

```
bleu_scores = []

for src, tgt in zip(english_sentences, german_sentences):
    pred = predict_sentence(model, src)

    score = calculate_bleu(tgt, pred)
    bleu_scores.append(score)

    print(f"\nInput: {src}")
    print(f"Target: {tgt}")
    print(f"Predicted: {pred}")
    print(f"BLEU: {score:.4f}")
```

```
Input: i love you
Target: ich liebe dich
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0235

Input: hello world
Target: hallo welt
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: how are you
Target: wie geht es dir
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: good morning
Target: guten morgen
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: thank you
Target: danke schoen
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: i am happy
Target: ich bin gluecklich
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0235

Input: he is good
Target: er ist gut
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: she is sad
Target: sie ist traurig
Predicted: ich ich ich ich ich ich ich ich ich ich
```

```
BLEU: 0.0000

Input: they are here
Target: sie sind hier
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0000

Input: i am learning
Target: ich lerne
Predicted: ich ich ich ich ich ich ich ich ich ich
BLEU: 0.0235
```

```
avg_bleu = np.mean(bleu_scores)

print("\nAverage BLEU Score:", avg_bleu)
```

```
Average BLEU Score: 0.007037248552467306
```

```
def predict_sentence_attn(model, sentence):

    seq = encode_sentence(sentence, src_vocab)
    seq = [SOS] + seq + [EOS]
    seq = seq + [PAD] * (10 - len(seq))
    seq = np.array([seq])

    encoder_outputs, hidden, cell = model.encoder(seq)

    dec_input = np.array([[SOS]])
    result = []

    for _ in range(10):
        preds = model.decoder(dec_input, hidden, cell, encoder_output

        pred_id = tf.argmax(preds[0, 0]).numpy()

        if pred_id == EOS:
            break

        result.append(tgt_vocab.idx2word.get(pred_id, "<unk>"))
        dec_input = np.array([[pred_id]])

    return " ".join(result)
```

```
def predict_sentence_no_attn(model, sentence):

    seq = encode_sentence(sentence, src_vocab)
    seq = [SOS] + seq + [EOS]
    seq = seq + [PAD] * (10 - len(seq))
    seq = np.array([seq])

    # Encoder
    _, hidden, cell = model.encoder(seq)
```

```
    dec_input = np.array([[SOS]])
    result = []

    for _ in range(10):

        preds = model.decoder(dec_input, hidden, cell)

        pred_id = tf.argmax(preds[0, 0]).numpy()

        if pred_id == EOS:
            break

        result.append(tgt_vocab.idx2word.get(pred_id, "<unk>"))

        dec_input = np.array([[pred_id]])

    return " " .join(result)
```

```
bleu_scores = []

for src, tgt in zip(english_sentences, german_sentences):
    pred = predict_sentence_attn(model, src)

    score = calculate_bleu(tgt, pred)
    bleu_scores.append(score)

print("BLEU (Attention):", np.mean(bleu_scores))
```

```
BLEU (Attention): 0.007037248552467306
```

```
bleu_no_attn = []

for src, tgt in zip(english_sentences, german_sentences):
    pred = predict_sentence_no_attn(model_no_attn, src)

    score = calculate_bleu(tgt, pred)
    bleu_no_attn.append(score)

print("BLEU (No Attention):", np.mean(bleu_no_attn))
```

```
BLEU (No Attention): 0.007037248552467306
```

Start coding or generate with AI.